

SESION 03

Registro, descubrimiento y ejecucion concurrente

Eureka Server, Config Client, dos instancias por microservicio y observabilidad dinamica con Prometheus y Loki.

Estudiante	Aldo Calla Ticona
Equipo	Sin grupo - trabajo individual
Rol / aporte	Implementacion y verificacion integral
Repositorio	github.com/Aldo-Ct/Pagalotu
Fecha	3 de septiembre de 2026

REGISTRO

4 UP

2 catalogo + 2 ordenes

METRICAS

4 UP

descubrimiento via Eureka

LOGS

2 streams

ordenes consultable en Loki

Fuente de verificacion: APIs reales de Config Server, Eureka, Prometheus y Loki ejecutadas bajo el usuario aldocallaticona.

Resumen y alcance verificado

Se completo la actividad guiada y la actividad autonoma de S03. pagatu-eureka recibe configuracion centralizada; catalogo y ordenes se registran por nombre logico; cuatro procesos atienden de forma concurrente; Prometheus descubre los targets mediante Eureka; y Promtail envia a Loki los archivos de ambos microservicios.

Criterio	Resultado	Evidencia
pagatu-eureka operativo	Cumplido	Health UP y API de registro
Ordenes como Config Client	Cumplido	Perfiles dev/prod por HTTP
Ordenes en Eureka	Cumplido	orden-ms:8082 y orden-ms:8083
Comprension del patron	Cumplido	Explicacion y defensa
Prometheus / Loki	Bonificacion	4 targets UP y logs de ordenes

Arquitectura comprobada

```
Config Server :18888
|
+--> Eureka :18761
|
+--> PAGATU-CATALOGO-MS :8080 / :8081
+--> ORDEN-MS :8082 / :8083
+--> Prometheus :19090 (Eureka SD)
Logs catalogo + ordenes --> Promtail --> Loki :13100
```

Metadatos de la ejecucion

Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46:20:56 -05

Evidencia 1 - pagatu-eureka operativo

La API oficial de Eureka mostro dos nombres logicos y cuatro instancias simultaneas. Los puertos son conocidos y cada instancia tiene su propio identificador.

GET http://localhost:18761/eureka/apps

Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05

Aplicacion	Instance ID	Puerto	Estado
PAGATU-CATALOGO-MS	pagatu-catalogo-ms:8080	8080	UP
PAGATU-CATALOGO-MS	pagatu-catalogo-ms:8081	8081	UP
ORDEN-MS	orden-ms:8083	8083	UP
ORDEN-MS	orden-ms:8082	8082	UP

Comprobacion de baja dinamica

A las 20:06 PET se detuvo catalogo 8081 con Ctrl+C. El cliente envio estado DOWN y completo el desregistro con HTTP 200. Eureka dejo una sola instancia de catalogo y Prometheus retiro automaticamente el target. Al reiniciar 8081, ambos sistemas recuperaron las cuatro instancias sin editar archivos de configuracion.

```
Unregistering application PAGATU-CATALOGO-MS with status DOWN
DiscoveryClient ... deregister status: 200
Resultado Eureka: pagatu-catalogo-ms:8080 (UP)
Reinicio 8081 -> pagatu-catalogo-ms:8081 (UP)
```

Evidencia 2 - ordenes como Config Client

El archivo local contiene solo el nombre de aplicacion, el perfil y la importacion de Config Server. Las propiedades de ambiente viven en config-repo y fueron recuperadas por HTTP.

GET /orden-ms/dev y GET /orden-ms/prod
Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05

Perfil	Puerto	Eureka defaultZone	Prometheus
dev	8082	http://localhost:18761/eureka	expuesto
prod	\${SERVER_PORT:8080}	http://pagatu-eureka:8761/eureka	expuesto

application.yml minimalista

```
spring:
  application:
    name: orden-ms
  profiles:
    active: dev
  config:
    import: optional:configserver:${CONFIG_SERVER_URL:http://localhost:18888}
```

En DEV se usa un instance-id determinista basado en nombre y puerto. En PROD se agrega un valor aleatorio para que las replicas Docker no se sobrescriban en Eureka aunque compartan el puerto interno 8080.

Evidencia 3 - dos instancias de ordenes

Las instancias 8082 y 8083 cargaron configuracion remota, conectaron con PostgreSQL, se registraron como ORDEN-MS y atendieron el mismo CRUD de forma independiente.

Pruebas HTTP y Maven ejecutadas en ambas instancias
Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05

Verificacion	8082	8083
/actuator/health	200 / UP	200 / UP
/api/v1/ordenes	200	200
/actuator/prometheus	200	200
/v3/api-docs	200	200
Eureka	orden-ms:8082 UP	orden-ms:8083 UP

Ejecucion

```
java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar --server.port=8082
java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar --server.port=8083
./mvnw -q test # exit 0
```

Las respuestas REST incluyeron X-Trace-ID distintos. Eso demuestra que la verificacion no se limito a observar procesos: cada puerto atendio una solicitud real contra la base de datos.

Evidencia opcional - Prometheus y Loki

Prometheus no contiene una lista manual de puertos. Consulta Eureka y descubre cualquier cliente registrado que exponga /actuador/prometheus.

GET :19090/api/v1/targets y consulta LogQL en :13100
Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05

Aplicacion	Instancia	Estado
PAGATU-CATALOGO-MS	pagatu-catalogo-ms:8080	UP
PAGATU-CATALOGO-MS	pagatu-catalogo-ms:8081	UP
ORDEN-MS	orden-ms:8083	UP
ORDEN-MS	orden-ms:8082	UP

Logs de ordenes recuperados de Loki

```
{application="pagatu-orden-ms"}
2026-08-31 20:07:51.726 [] DEBUG o.s.web.client.DefaultRestClient - Reading to
[com.netflix.discovery.shared.A
2026-08-31 20:07:50.529 [d8ffcdab-f4e3-450b-98e9-0cf1da361e31] DEBUG
o.s.web.servlet.DispatcherServlet - Compl
2026-08-31 20:07:50.528 [d8ffcdab-f4e3-450b-98e9-0cf1da361e31] DEBUG
o.s.w.s.m.a.HttpEntityMethodProcessor -
2026-08-31 20:07:50.528 [d8ffcdab-f4e3-450b-98e9-0cf1da361e31] DEBUG
o.s.w.s.m.a.HttpEntityMethodProcessor -
2026-08-31 20:07:51.849 [] DEBUG o.s.web.client.DefaultRestClient - Reading to
[com.netflix.discovery.shared.A
```

Loki devolvió 2 streams para ordenes. Promtail diferencia servicios mediante la etiqueta application y monta las carpetas logs de catalogo y ordenes en modo solo lectura.

Comprension del patron y hallazgo

Por que ya no se necesita una lista de direcciones

Un consumidor trabaja con el nombre logico ORDEN-MS. Eureka conserva la relacion actual entre ese nombre y 8082/8083; el consumidor consulta el registro y un cliente con balanceo selecciona una instancia. Los puertos pueden seguir siendo fijos: lo importante es que la lista no esta incrustada en el consumidor, sino administrada dinamicamente por el registro.

Error o hallazgo real

Al detener catalogo 8081, Prometheus no lo mantuvo indefinidamente como DOWN. El target desaparecio tras el refresco porque el cliente se desregistro gracefully de Eureka. El diagnostico se confirmo observando el HTTP 200 del deregister y la lista reducida de Eureka. Al reiniciar la instancia, reaparecio sin modificar prometheus-dev.yml.

Reflexion tecnica

El Gateway de S4 necesita una referencia estable para servicios cuyo numero de instancias puede cambiar. Eureka aporta nombres logicos y un inventario actualizado. El Gateway podra resolver ORDEN-MS o PAGATU-CATALOGO-MS sin almacenar cada direccion. Esa informacion permite repartir solicitudes y excluir instancias retiradas. Por eso registro y descubrimiento son prerequisites directos del enrutamiento y balanceo de carga.

Preguntas de defensa

1	Eureka no se registra a si mismo porque es el servidor del registro, no una instancia consumidora.
2	Sin el override, la segunda instancia de catalogo intentaria ocupar 8080 y fallaria por puerto en uso.
3	Ordenes se verifico en la API de Eureka y mediante peticiones independientes a 8082 y 8083.
4	Sin heartbeat, Eureka termina removiendo la instancia; con apagado graceful, el desregistro es inmediato.
5	spring.application.name selecciona configuracion en Config Server y define el nombre logico en Eureka.
6	Prometheus tambien consume Eureka para descubrir targets sin direcciones escritas a mano.

Captura 1 - Registro en Eureka

Captura original: 01_eureka_cuatro_instancias.png
Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05

System Status

Environment	test	Current time	2026-09-03T20:46:14 -0500
Data center	default	Uptime	00:10
		Lease expiration enabled	true
		Renews threshold	8
		Renews (last min)	16

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDEN-MS	n/a (2)	(2)	UP (2) - orden-ms:8083 , orden-ms:8082
PAGATU-CATALOGO-MS	n/a (2)	(2)	UP (2) - pagatu-catalogo-ms:8080 , pagatu-catalogo-ms:8081

General Info

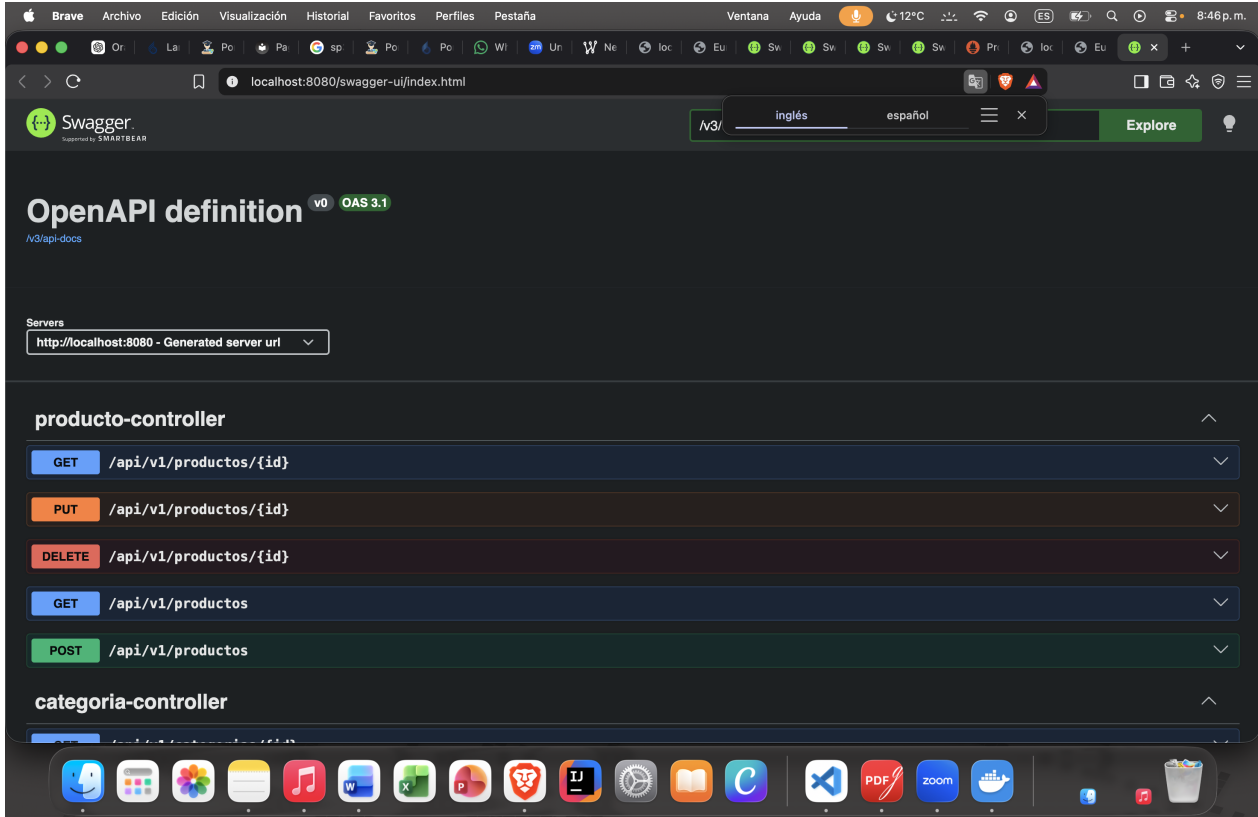
Eureka muestra dos instancias UP de ORDEN-MS (8082 y 8083) y dos de PAGATU-CATALOGO-MS (8080 y 8081).

La barra de direcciones y el reloj del sistema permanecen visibles para identificar el endpoint local y el momento de la verificación.

Captura 2 - Swagger de catalogo 8080

Captura original: 02_catalogo_8080_swagger.png

Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05



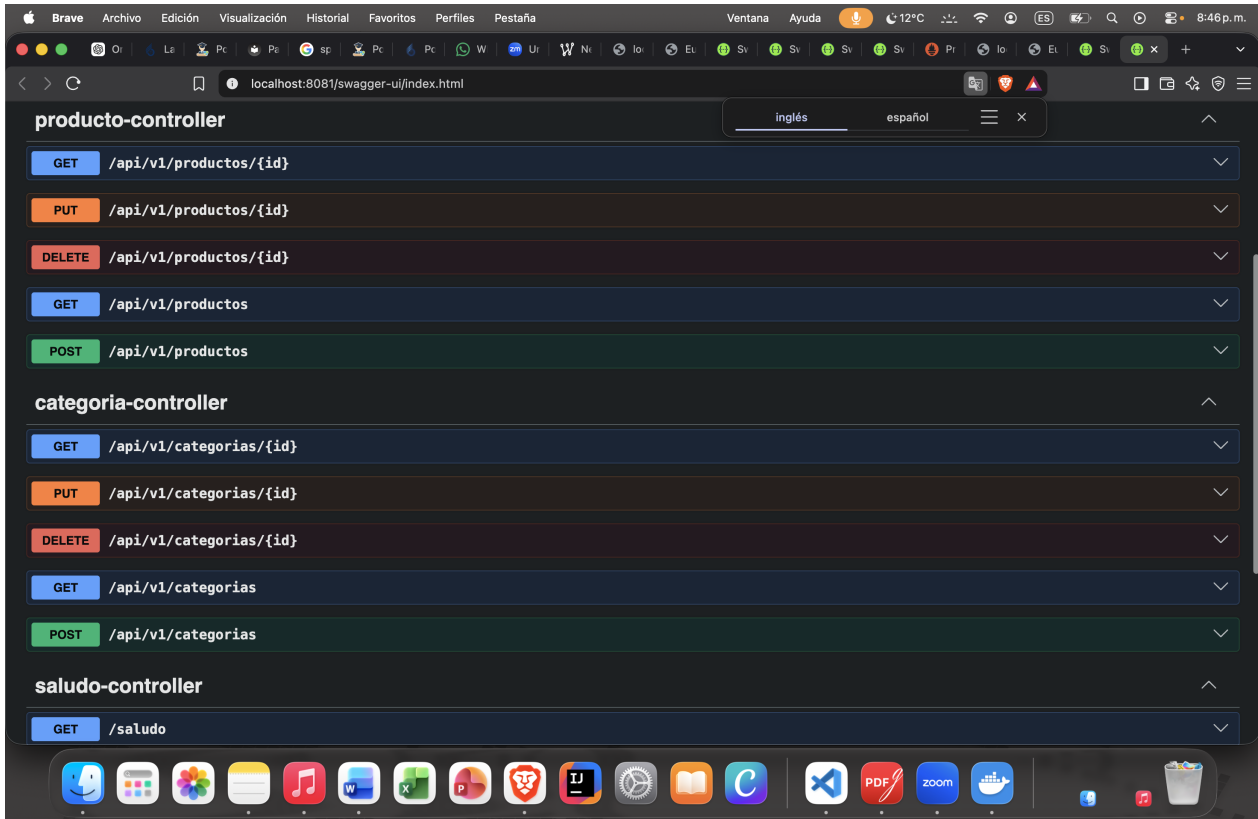
La primera instancia de catalogo publica el CRUD de productos y categorias.

La barra de direcciones y el reloj del sistema permanecen visibles para identificar el endpoint local y el momento de la verificación.

Captura 3 - Swagger de catalogo 8081

Captura original: 03_catalogo_8081_swagger.png

Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05



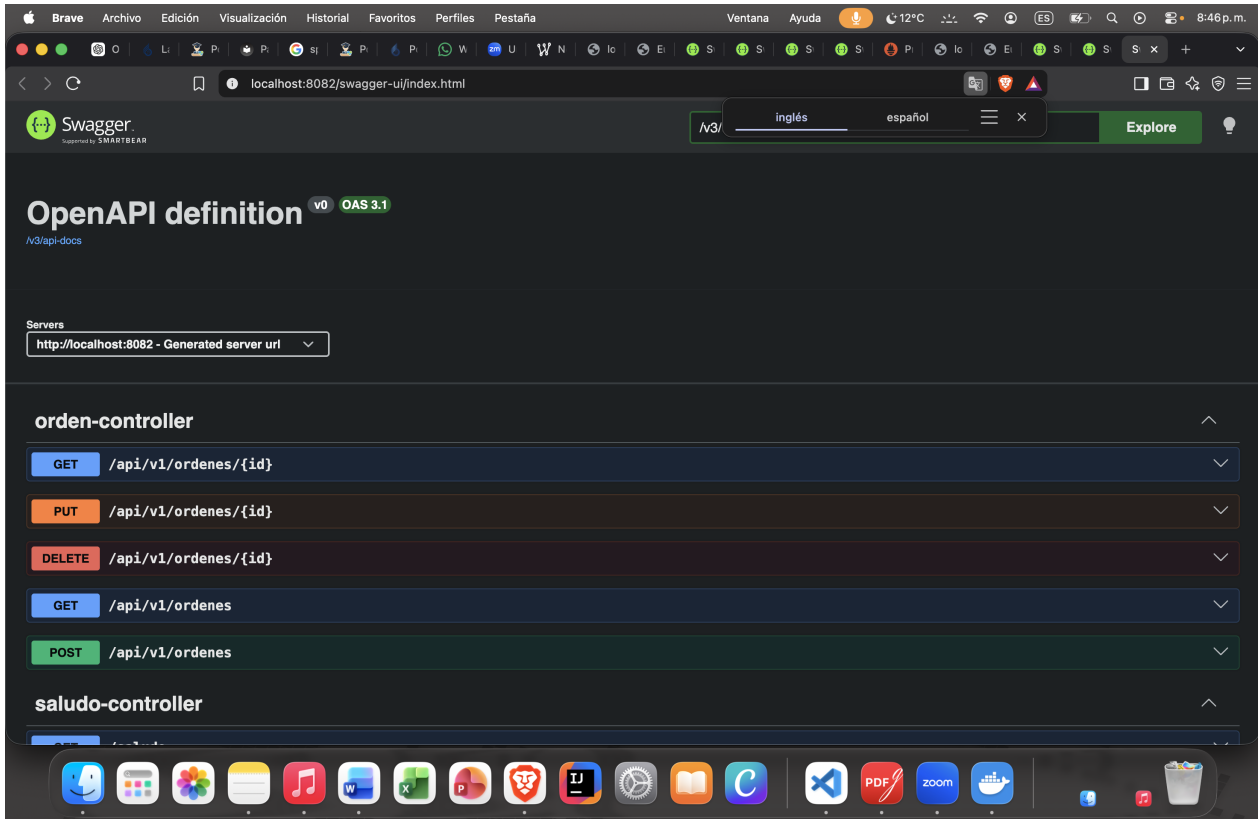
La segunda instancia publica la misma API desde un puerto independiente.

La barra de direcciones y el reloj del sistema permanecen visibles para identificar el endpoint local y el momento de la verificación.

Captura 4 - Swagger de ordenes 8082

Captura original: 04_orden_8082_swagger.png

Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05



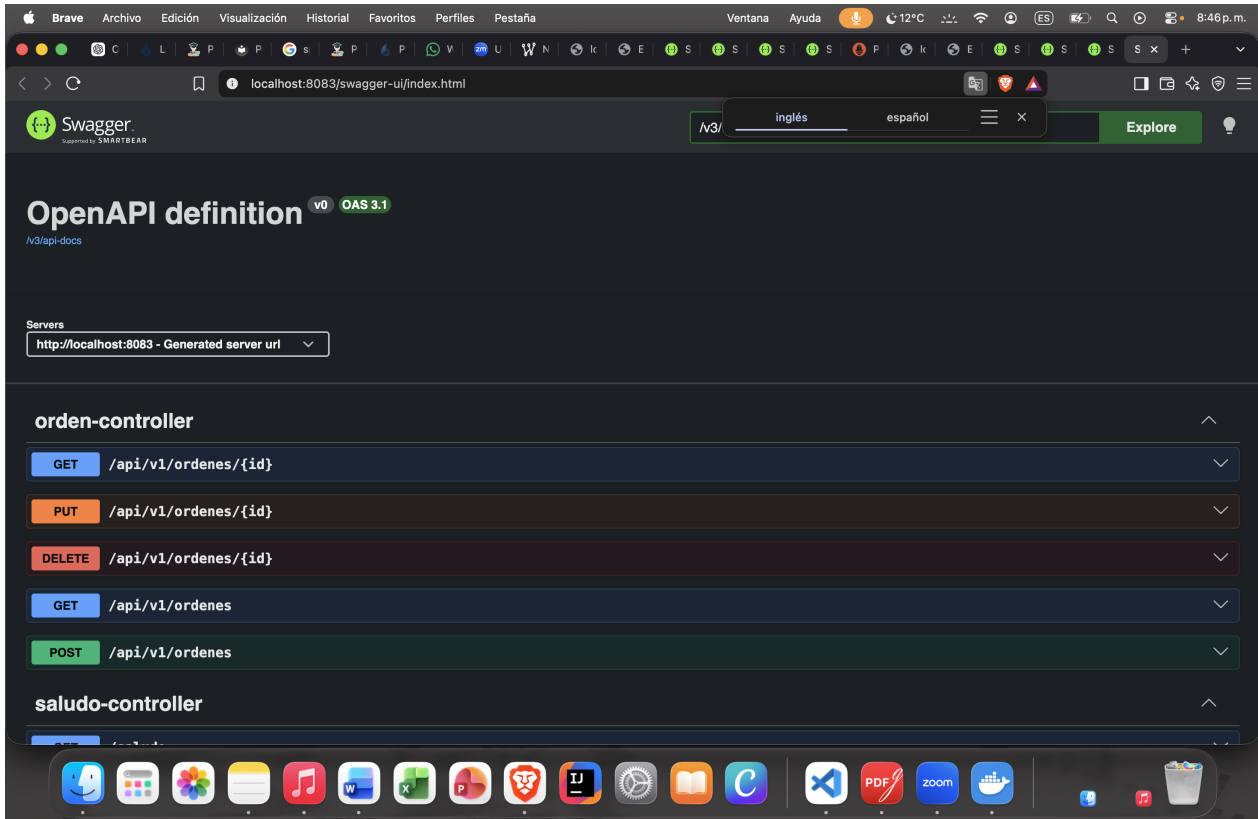
La primera instancia de ordenes expone el CRUD y el endpoint /saludo.

La barra de direcciones y el reloj del sistema permanecen visibles para identificar el endpoint local y el momento de la verificación.

Captura 5 - Swagger de ordenes 8083

Captura original: 05_orden_8083_swagger.png

Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05

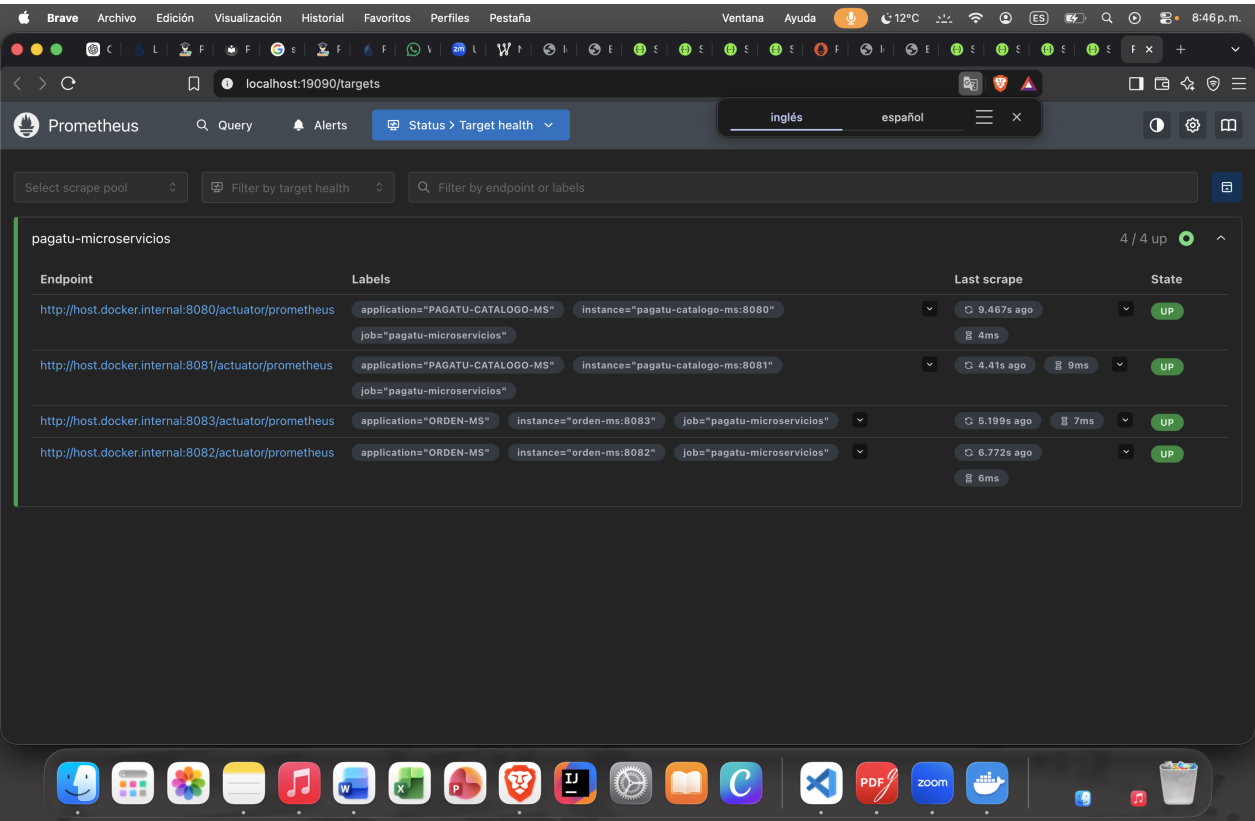


La segunda instancia de ordenes atiende la misma API concurrentemente.

La barra de direcciones y el reloj del sistema permanecen visibles para identificar el endpoint local y el momento de la verificación.

Captura 6 - Descubrimiento en Prometheus

Captura original: 06_prometheus_targets.png
Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05



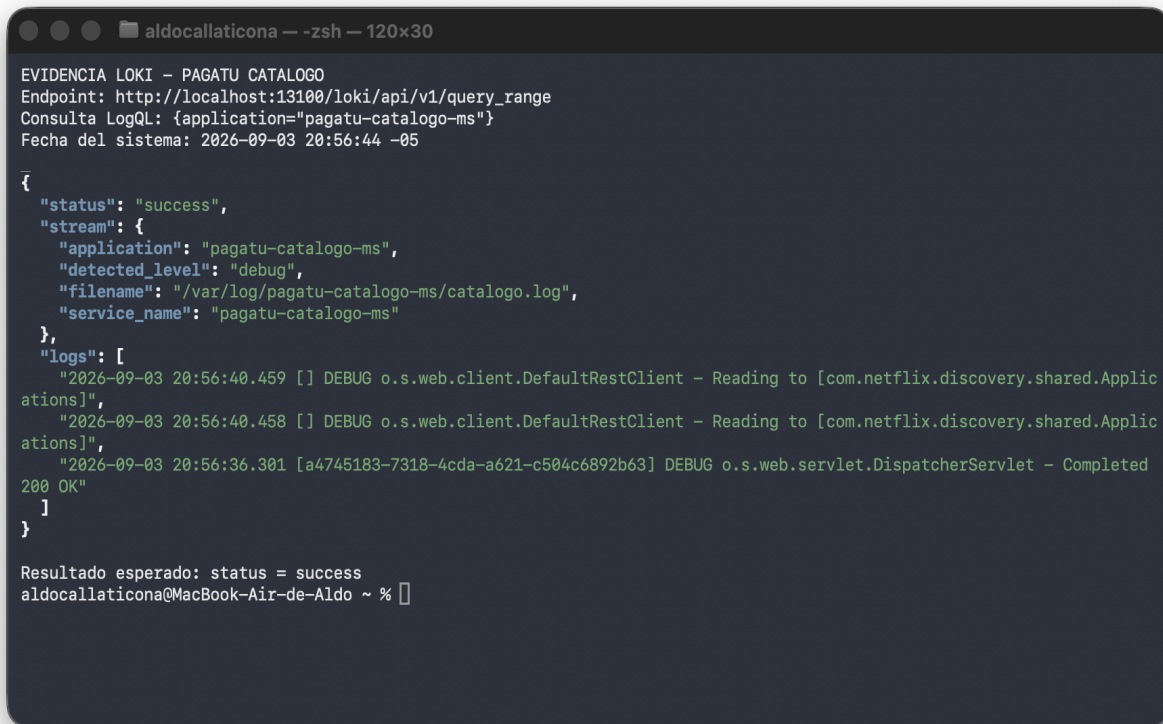
Target health confirma 4/4 targets UP descubiertos desde Eureka.

La barra de direcciones y el reloj del sistema permanecen visibles para identificar el endpoint local y el momento de la verificación.

Captura 7 - Consulta real de catalogo en Loki

Captura original: 07_loki_terminal_catalogo.png

Usuario: aldocallicona | Fecha/hora del sistema: 2026-09-03 20:46-20:56 -05

A terminal window titled 'aldocallicona -- zsh -- 120x30' displays a Loki query and its response. The query is for 'EVIDENCIA LOKI - PAGATU CATALOGO' with endpoint 'http://localhost:13100/loki/api/v1/query_range' and LogQL '{application="pagatu-catalogo-ms"}'. The response is a JSON object with status 'success', stream information, and a list of logs. The logs show three entries: two 'DEBUG' messages from 'o.s.web.client.DefaultRestClient' and one 'DEBUG' message from 'o.s.web.servlet.DispatcherServlet' with a '200 OK' status.

```
aldocallicona -- zsh -- 120x30

EVIDENCIA LOKI - PAGATU CATALOGO
Endpoint: http://localhost:13100/loki/api/v1/query_range
Consulta LogQL: {application="pagatu-catalogo-ms"}
Fecha del sistema: 2026-09-03 20:56:44 -05

{
  "status": "success",
  "stream": {
    "application": "pagatu-catalogo-ms",
    "detected_level": "debug",
    "filename": "/var/log/pagatu-catalogo-ms/catalogo.log",
    "service_name": "pagatu-catalogo-ms"
  },
  "logs": [
    "2026-09-03 20:56:40.459 [] DEBUG o.s.web.client.DefaultRestClient - Reading to [com.netflix.discovery.shared.Applications]",
    "2026-09-03 20:56:40.458 [] DEBUG o.s.web.client.DefaultRestClient - Reading to [com.netflix.discovery.shared.Applications]",
    "2026-09-03 20:56:36.301 [a4745183-7318-4cda-a621-c504c6892b63] DEBUG o.s.web.servlet.DispatcherServlet - Completed 200 OK"
  ]
}

Resultado esperado: status = success
aldocallicona@MacBook-Air-de-Aldo ~ %
```

La terminal confirma status success, la etiqueta application=pagatu-catalogo-ms y logs reales del microservicio.

La terminal conserva visibles el endpoint consultado, la expresion LogQL, la fecha del sistema y el resultado devuelto por Loki.

Anexo - Feedback de la sesion

1	El registro convierte direcciones fisicas en un nombre logico estable y actualizable.
2	Diferenciar un target DOWN de uno retirado cuando Eureka deja de anunciarlo.
3	Como combinara el Gateway la resolucion por nombre con Spring Cloud LoadBalancer.
4	Entendido - lo domino y podria explicarlo.
5	Comparar en S4 una ruta fija con una ruta lb:// observando la instancia elegida.
6	Muy comprometido - se completo tambien el bloque opcional de observabilidad.
7	10/10

Trazabilidad de evidencia

Los valores del informe y las capturas se obtuvieron de ejecuciones reales verificadas el 3 de septiembre de 2026 bajo el usuario aldocallicona. Tambien se conservaron los extractos JSON de Config Server, Eureka, Prometheus y Loki usados para generar las tablas.

Las siete capturas del anexo provienen de los servicios activos del proyecto: Eureka, Swagger en los cuatro puertos, Prometheus y la consulta HTTP de Loki. Los extractos de API permiten contrastar visualmente los mismos resultados.

Guia de referencia: 262dist.github.io/pagatu/sesiones/S03_Registro_Descubrimiento_Ejecucion_Concurrente/